Page 1: What is Zephyr RTOS?

Introduction, Purpose, Architecture & References

1.1 Understanding Real-Time Operating Systems (RTOS)

An **RTOS (Real-Time Operating System)** is an operating system engineered to execute instructions with strict, predictable timing. While general-purpose operating systems like Linux or Windows focus on overall data throughput, an RTOS guarantees that time-critical events, hardware interrupts, and sensor data are processed deterministically within exact deadlines.

1.2 About the Zephyr Project

Zephyr is an open-source, vendor-neutral real-time operating system managed by the **Linux Foundation**, licensed under the permissive **Apache 2.0** license. Backed by industry giants including **Microchip, Google, Intel, Nordic Semiconductor, NXP, STMicroelectronics, and Qualcomm**, Zephyr scales from small 32-bit microcontrollers with only 8 KB of RAM up to multi-core MPUs.

Core Architectural Pillars of Zephyr:

- **Preemptive Real-Time Kernel:** Deterministic priority-based scheduling, tickless idle mode, semaphores, mutexes, message queues, and memory slabs.
- **Devicetree (DTS):** Hardware abstraction borrowed from Linux, decoupling silicon pinouts from C code.
- **Kconfig Configuration:** Granular compile-time feature selection to strip unused code and save Flash/RAM.
- **Batteries-Included Connectivity:** Native, certified Bluetooth Low Energy (BLE), Wi-Fi, Ethernet, CAN-FD, USB, and OpenThread stacks.

1.3 Why Zephyr for Embedded Systems Education?

Traditional embedded curricula lock students into vendor-specific bare-metal registers. Zephyr equips students with universal, cross-architecture engineering skills used in modern automotive, medical, industrial, and IoT products.

1.4 Official Referral & Reference Links

Resource	URL Link	Description
Zephyr Project Home	**https://zephyrproject.org/**	Official website, release notes, and community links
Zephyr Documentation	**https://docs.zephyrproject.org/**	Comprehensive APIs, driver guides, and sample codes
Linux Foundation Page	**https://www.linuxfoundation.org/**	Governance, charter, and open-source compliance
Zephyr GitHub Repository	**https://github.com/zephyrproject-rtos/zephyr**	Upstream kernel source code, issues, and pull requests

Page 2: Microchip & Zephyr Integration

How Microchip Brings Upstream Zephyr to Silicon Boards

2.1 The Microchip & Zephyr Partnership

Microchip Technology is an active member and primary silicon contributor to the Zephyr Project. Rather than requiring users to maintain out-of-tree vendor patches, Microchip directly integrates official silicon drivers, clock generators, and Device Family Packs (DFP) into the upstream Zephyr repository.

2.2 Layered Board Porting Architecture

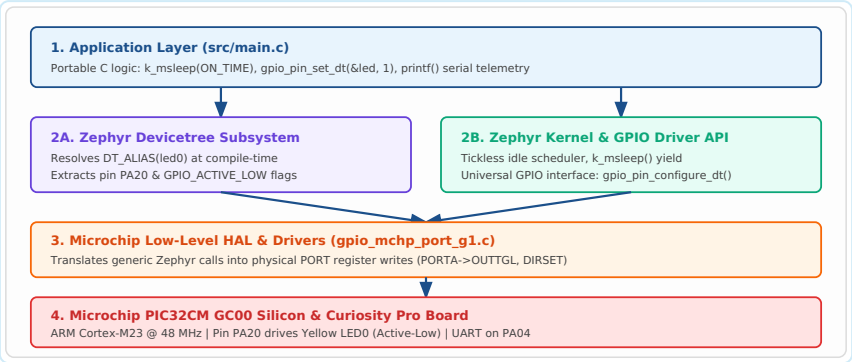


Figure 2.1: Microchip Upstream HAL & Zephyr Driver Layering.

Microchip brings Zephyr support to hardware boards through four synchronized layers:

- **1. Microchip HAL Module (`modules/hal/microchip`):** Contains vendor register header files, CMSIS startup vectors, and Device Family Packs (DFP) for PIC32C/SAM devices.
- **2. SoC Series Architecture (`zephyr/soc/microchip/pic32c`):** Configures the 48 MHz FDPLL96M oscillator, Flash Wait States (NVMCTRL), MPU regions, and Power Management.
- **3. Device Driver Layer (`zephyr/drivers`):** Standardized drivers for Microchip PORT (GPIO), SERCOM (UART/I2C/SPI), and ADC peripherals.
- **4. Board Target Definition (`zephyr/boards/microchip/pic32c/pic32cm_gc00_cpro`):** Declares the pinout, chosen serial console, memory partitions, and onboard LEDs/buttons via Devicetree (`.dts`).

2.3 Official Microchip Referral & Source Links

Component	Official URL / Referral Link	Repository Path
Microchip Upstream HAL	**https://github.com/zephyrproject-rtos/hal_microchip**	`modules/hal/microchip`
Microchip 32-bit MCUs	**https://www.microchip.com/pic32**	Official Silicon Portal
PIC32CM GC00 Board Docs	**https://docs.zephyrproject.org/.../pic32cm_gc00_cpro**	`boards/microchip/pic32c/pic32cm_gc00_cpro/`

Page 3: Board Specifications & Peripherals

Hardware Specs & Peripheral Support for PIC32CM GC00 Curiosity Pro

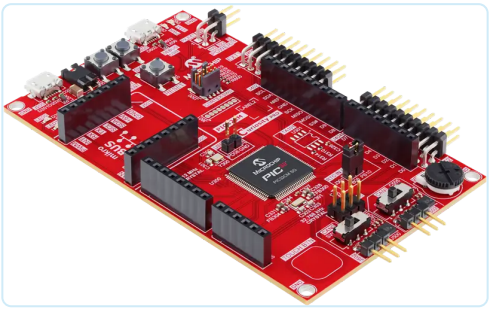


Figure 3.1: Microchip PIC32CM GC00 Curiosity Pro Evaluation Kit.

3.1 Hardware Specifications

Parameter	Specification Value	Details & Benefits
Microcontroller	PIC32CM5112GC00100	Microchip 32-bit MCU (100-pin TQFP)
Processor Core	ARM Cortex-M23	ARMv8-M Baseline RISC architecture
System Frequency	48.0 MHz FDPLL96M	Internal high-precision clock generator
Flash / SRAM	512 KB Flash / 128 KB SRAM	Zero-wait-state high speed execution
Onboard Debugger	Microchip nEDBG	CMSIS-DAP debugger & CDC Virtual COM Port
Operating Voltage	3.3 V DC	Supplied via USB or external header

3.2 Peripheral Support Status in Zephyr

Peripheral	Driver / Node Identifier	Status	Hardware Mapping
GPIO Controller	`gpio_mchp_port_g1` (PORTA, PORTB)	**Supported**	PA20 (LED0), PA18 (LED1), PB07 (SW0), PB11 (SW1)
UART / Console	`uart_mchp_sercom`	**Supported**	PA04 / PA05 (115200 8N1 to EDBG Virtual COM)
Pin Controller	`pinctrl-pic32c`	**Supported**	Dynamic pin multiplexing subsystem
Core Timer	ARM Cortex-M SysTick	**Supported**	Kernel 1000 Hz system tick for `k_msleep()`
Flash Controller	`nvmctrl`	**Supported**	Boot flash (16KB) & Code partition (496KB)
ADC / DAC / I2C	Microchip SERCOM / ADC	Roadmap	Available through external overlays

Page 4: Development Platform

Hardware and Software Architecture Overview

01 Hardware

- **Board:** Microchip PIC32CM GC00 Curiosity Pro (EV33A17A)
- **MCU:** PIC32CM5112GC00100 (32-bit ARM Cortex-M23 @ 48 MHz)
- **Onboard LED:** Yellow LED0 (Pin PA20, Active-Low) for visual output
- **User Pushbuttons:** Mechanical switch SW0 (Pin PB07) for interactive input
- **USB Interface:** Micro-USB connection (J100) for programming, debugging, and serial UART telemetry

02 Software

- **Development IDE:** MPLAB VS Code / Visual Studio Code with Zephyr extensions
- **Operating System:** Zephyr RTOS Workspace (v3.7+)
- **Build Tool:** West meta-tool (CMake & Ninja automated workflow)
- **Compiler Toolchain:** Compatible Zephyr ARM SDK (`arm-zephyr-eabi`)
- **Board Support Package:** Upstream `pic32cm_gc00_cpro` board target

Outcome: A reliable, standardized environment for developing, building, flashing, and running professional Zephyr RTOS applications on Microchip hardware.

Page 5: Cloning & Local Setup

How to Clone Zephyr and Initialize Your Local Workspace

5.1 Workspace Folder Architecture

Zephyr projects are managed using a workspace structure where repositories and modules are coordinated through `west`.

5.2 Step-by-Step Terminal Instructions

Step 1: Create a Dedicated Directory & Virtual Environment

```
# Create workspace root directory
mkdir -p ~/zephyrproject && cd ~/zephyrproject

# Create and activate Python 3 virtual environment
python3 -m venv .venv
source .venv/bin/activate
```

Step 2: Install West (Zephyr's Meta-Tool)

```
pip install --upgrade pip
pip install west
```

Step 3: Initialize the Zephyr Manifest Repository

```
# Initialize west in the local workspace directory
west init -l zephyr
```

Step 4: Clone External Submodules & Microchip Packs

```
# Clones hal_microchip, CMSIS packs, mbedTLS, and tools
west update
```

Step 5: Export CMake Packages & Python Dependencies

```
# Register Zephyr CMake packages
west zephyr-export

# Install required Python packages for Zephyr tools
pip install -r zephyr/scripts/requirements.txt
```

> **Pro Tip for Students:** Always ensure your virtual environment is active before running any West commands: `source ~/zephyrproject/.venv/bin/activate`

Page 6: Environment & Toolchain Setup

Installing SDK Toolchain, Dependencies & USB Permissions

6.1 Host Operating System Dependencies (Ubuntu / WSL2)

```
sudo apt update && sudo apt install -y git cmake ninja-build gperf ccache dfu-util   device-tree-
compiler wget python3-dev python3-pip python3-setuptools xz-utils make gcc
```

6.2 Installing the Zephyr ARM SDK Cross-Compiler

The **Zephyr SDK** contains cross-compilation toolchains (`arm-zephyr-eabi-gcc`) required to build ARM Cortex-M binaries on an x86 host PC:

```
# 1. Download Zephyr SDK archive
cd ~
wget https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.16.8/zephyr-sdk-0.16.8_linux-
x86_64.tar.xz

# 2. Extract archive
tar xvf zephyr-sdk-0.16.8_linux-x86_64.tar.xz

# 3. Run toolchain installation script for ARM
cd zephyr-sdk-0.16.8
./setup.sh -t arm-zephyr-eabi -h -c
```

6.3 Configuring USB Permissions (udev Rules)

Linux restricts direct USB access to root by default. Installing Zephyr's udev rules permits regular user access to the onboard Microchip nEDBG programmer:

```
sudo cp ~/zephyrproject/zephyr/scripts/udev/rules.d/* /etc/udev/rules.d/
sudo udevadm control --reload-rules
sudo udevadm trigger
```

6.4 Verifying the Environment

Verification Command	Expected Response
`west --version`	West version 1.2.0 or newer
`arm-zephyr-eabi-gcc --version`	arm-zephyr-eabi-gcc 12.2.0 (Zephyr SDK)
`cmake --version`	CMake version 3.20.0 or higher

Page 7: Application Configuration Files

Deep Dive into CMakeLists.txt, prj.conf, and app.overlay

7.1 CMakeLists.txt — Build Definition

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(pic32cm_gc00_blinky)
target_sources(app PRIVATE src/main.c)
```

`find_package(Zephyr)` resolves board Devicetree files, configures Kconfig options, imports the ARM toolchain, and links the Zephyr RTOS kernel into the executable.

7.2 prj.conf — Kconfig Kernel & Driver Configuration

```
CONFIG_GPIO=y               # Compiles the Microchip PORT GPIO driver
CONFIG_PRINTK=y             # Enables Zephyr's formatted logging system
CONFIG_STDOUT_CONSOLE=y     # Directs C printf() output to the UART serial port
```

In Zephyr, drivers are disabled by default to save flash memory. `prj.conf` enables only what your application requires.

7.3 app.overlay — Custom Devicetree Overlays

An **overlay file** (`app.overlay`) allows students to modify or add hardware nodes without modifying the main board Devicetree:

```
/ {
    aliases {
        my-custom-led = &custom_led;
    };
    leds {
        compatible = "gpio-leds";
        custom_led: led_custom {
            gpios = <&porta 18 GPIO_ACTIVE_LOW>;
            label = "Secondary Yellow LED1";
        };
    };
};
```

Devicetree Rule: Devicetree maps hardware topology (Pins → Aliases). C code accesses peripherals via generic aliases like `DT_ALIAS(led0)`, keeping firmware 100% portable.

Page 8: Building & Flashing Firmware

Compilation Pipeline, West Commands & Flashing Guide

8.1 The Zephyr Compilation Workflow

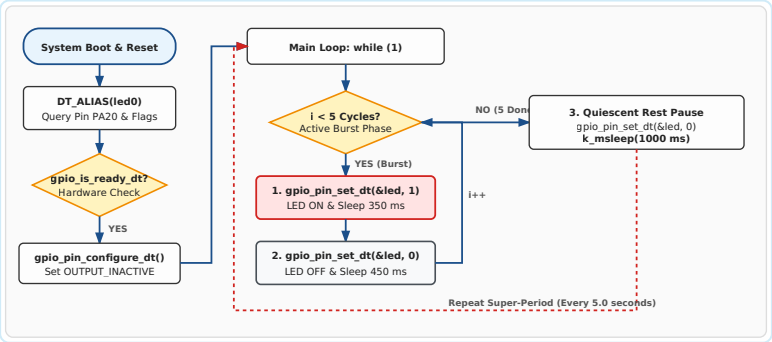


Figure 8.1: Complete Build & Execution Pipeline.

8.2 Step-by-Step Command Matrix

Action	Command Line	Description
Clean Pristine Build	`west build -b pic32cm_gc00_cpro pic32cm_gc00_blinky -p always`	Discards prior cache, generates new Devicetree headers, compiles all files
Incremental Build	`west build -p auto`	Fast compilation recompiling only modified source files
Flash Hardware	`west flash`	Invokes J-Link / EDBG to program MCU internal flash over USB
Flash with J-Link	`west flash --runner jlink`	Explicitly specifies J-Link runner for Curiosity Pro

8.3 Generated Build Artifacts

- `build/zephyr/zephyr.elf`: Complete ELF binary containing symbols and debug tables.
- `build/zephyr/zephyr.hex`: Intel HEX record used by hardware programmers.
- `build/zephyr/zephyr.bin`: Raw flat machine code image programmed into internal Flash memory.

Page 9: Serial Telemetry & Timing

UART Output Logs, Timing Waveform & Electrical Logic

9.1 Serial Console Connection (115200 8N1)

Connect to the USB CDC Virtual COM port exposed by the onboard debugger:

```
# Connect via picocom on Linux
picocom -b 115200 /dev/ttyACM0
```

9.2 Expected Serial Output Log

```
*** Booting Zephyr OS build v3.7.0 ***
Iteration 1/5 - LED state: ON (350 ms ON)
Iteration 1/5 - LED state: OFF (450 ms OFF)
Iteration 2/5 - LED state: ON (350 ms ON)
Iteration 2/5 - LED state: OFF (450 ms OFF)
Iteration 3/5 - LED state: ON (350 ms ON)
Iteration 3/5 - LED state: OFF (450 ms OFF)
Iteration 4/5 - LED state: ON (350 ms ON)
Iteration 4/5 - LED state: OFF (450 ms OFF)
Iteration 5/5 - LED state: ON (350 ms ON)
Iteration 5/5 - LED state: OFF (450 ms OFF)
Pause - LED state: OFF (1000 ms Pause)

Iteration 1/5 - LED state: ON (350 ms ON)
...
```

9.3 Accurate Timing Waveform Analysis

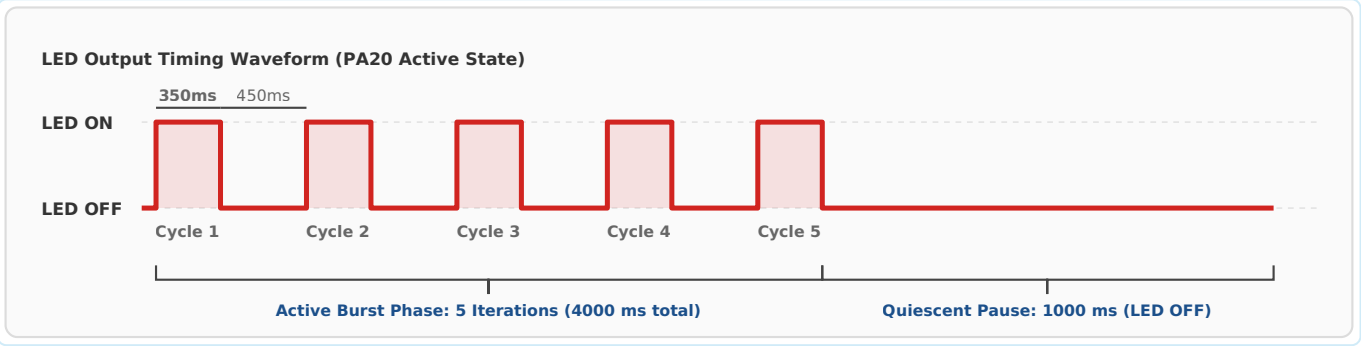


Figure 9.1: Deterministic 5-Burst Active Pulse & 1000 ms Quiescent Rest Waveform.

Page 10: Conclusion & Lab Exercises

Summary, Student Exercises & Official References

10.1 Key Learning Summary

- **Standardized Driver Interfaces:** The application calls `gpio_pin_set_dt()` and `k_msleep()`, which run identically on any Zephyr-supported microcontroller.
- **Electrical Logic Abstraction:** Zephyr automatically translates logical states into physical voltage levels based on Devicetree polarity flags (`GPIO_ACTIVE_LOW`).
- **Tickless Power Savings:** `k_msleep()` allows the ARM Cortex-M23 core to enter low-power sleep between LED pulses.

10.2 Student Lab Challenges

Challenge 1: Morse Code SOS Transmitter

Modify the delay constants to transmit international Morse code SOS on LED0 (3 short pulses of 150 ms, 3 long pulses of 450 ms, 3 short pulses of 150 ms, followed by a 2000 ms pause).

Challenge 2: Dual Alternating Beacon Flasher

Add secondary LED1 on pin PA18 using `DT_ALIAS(led1)`. Alternate LED0 and LED1 like an emergency responder light bar.

Challenge 3: Pushbutton-Gated Pattern

Read mechanical switch SW0 on pin PB07 using `DT_ALIAS(sw0)`. Only execute the 5-burst blinking sequence when SW0 is held down.

10.3 Official References & Links

Portal	URL
Microchip Official	**https://www.microchip.com**
Zephyr Project	**https://zephyrproject.org**
Project GitHub	**https://github.com/sujandurai/pic32cm_gc00_blinky**

Congratulations on mastering Zephyr RTOS fundamentals on Microchip PIC32CM hardware!